

1. PROJECT TITLE

Pocket Packet Sniffer

Group Number: 4

2. TEAM PROFILE

Sl. No.	USN	SRN / PRN	Student Name
1	PES1UG24CS066	PES1202402282	Ankita S [Team Lead]
2	PES1UG24CS073	PES1202402250	Anusha Gupta
3	PES1UG24CS125	PES1202400244	Charan M
4	PES1UG25CS852	PES1202503586	Vaghasiya Akshar Arvindbhai

3. PROPOSED PROJECT DESCRIPTION

3.1 Background and Problem

Inspecting live network traffic today usually means connecting a laptop running a tool such as Wireshark or tcpdump to the network segment of interest. This requires the right drivers, capture permissions, and enough familiarity with the tool to make sense of the raw output. For quick, informal checks — for example, during a network lab session, a small audit, or while debugging an IoT/embedded network — carrying and configuring a full laptop setup is inconvenient, and the raw packet-level output is not easy to read at a glance.

This system is primarily intended to support networking and cybersecurity laboratory sessions, student learning, and small-scale network troubleshooting where a portable traffic-monitoring device can provide a quick overview of network activity.

3.2 Problem Being Addressed

There is no small, self-contained, battery-friendly device that a student or junior network technician can carry to a network point, start a capture, and get an immediate, readable summary of what is happening on that segment — along with a basic flag for obviously suspicious activity — without first setting up capture software on a full computer.

3.3 Proposed System

The **Pocket Packet Sniffer** is a compact device built around a Raspberry Pi (or equivalent single-board computer), with an ESP32 available as an auxiliary hardware component where the final design requires it. It captures live packets from a chosen network interface, runs them through **Zeek** — an established open-source network security monitoring engine — for protocol parsing and behavioural analysis, and presents the results through a lightweight Python-based interface. Captured sessions can be stored locally and exported for deeper offline analysis on a full workstation if needed.

3.4 High-Level Working

1. The device uses a Raspberry Pi-based network capture interface to obtain traffic from the target network segment. An ESP32 may be used as an auxiliary hardware component where required by the final hardware design. The captured traffic is then forwarded to Zeek for protocol analysis and monitoring.
2. Zeek, running on the Raspberry Pi, parses protocols and produces structured logs (e.g. conn.log, dns.log, http.log) and applies its detection scripts to flag anomalies.
3. A Python-based control application reads Zeek's output in real time and renders a simplified, human-readable summary and live statistics.
4. Captured sessions (raw packet captures and/or Zeek logs) are stored locally and can be exported for further analysis, e.g. by opening the .pcap file in Wireshark.

3.5 New Project or Extension

This is a **new project** designed and built by the team. It integrates an established open-source analysis engine (Zeek) rather than reimplementing protocol parsing from scratch, which is standard practice for a project of this scope; the team's original contribution is the capture pipeline, the portable hardware integration, the summarisation/alerting logic on top of Zeek, and the session management layer.

3.6 Intended Users

Networking and cybersecurity students, teaching assistants running network/security lab sessions, and junior IT or network administrators who need a quick, low-effort way to check traffic on a segment without full Wireshark/Zeek expertise on-site.

3.7 Intended Customer / Stakeholder

Since this is an academic mini-project, there is no external paying customer. The primary stakeholders are networking and cybersecurity students, teaching staff, and laboratory instructors who may use the system for educational demonstrations, controlled experiments, and basic network traffic analysis.

3.8 Why the System is Useful

Portability and simplified, readable output lower the effort needed to get a first look at network activity, compared to setting up a full laptop-based capture tool at the point of interest.

3.9 Scope

In scope: capturing live traffic from one interface at a time; parsing it using Zeek's built-in protocol analysers; presenting summarised statistics (protocol breakdown, top talkers, connection counts); alerting on predefined suspicious patterns already covered by Zeek's standard/extended scripts (e.g. port-scan behaviour); local storage and export of sessions.

Out of scope: this project is not intended to be a replacement for Wireshark, a SIEM, or a full enterprise network monitoring platform. It does not perform unrestricted deep packet inspection and is not a universal cyberattack detection system — correlation across multiple devices/sites and custom signatures beyond what Zeek's default/extended scripts provide are outside the scope of this mini-project.

3.10 Project Objectives

- Develop a compact and portable network traffic monitoring system using a Raspberry Pi-based platform.
- Capture and analyse network traffic using an established network monitoring engine such as Zeek.
- Present network activity through a simplified real-time dashboard.
- Identify and report selected suspicious traffic patterns supported by the analysis engine.
- Provide local storage and export of captured sessions for later analysis.

- Develop the system using a modular and maintainable architecture following Agile software engineering practices.

4. FUNCTIONAL FEATURES

The following are the major user-facing functionalities of the system. Each is a complete, independently demonstrable capability — not a technical task.

4.1 Live Capture & Session Control

User action: Selects a network interface, starts or stops a capture session, and optionally sets a duration or a basic filter (e.g., by protocol).

System behaviour: Initiates packet capture on the chosen interface, manages the session lifecycle, streams captured packets into Zeek in real time, and writes the raw capture (.pcap) to local storage.

Outcome: A clearly bounded capture session exists, with a raw packet capture and corresponding Zeek logs ready for the next stages.

4.2 Real-Time Traffic Summary Dashboard

User action: Views a live-updating summary while a capture is running or after it ends.

System behaviour: Continuously reads Zeek's structured logs, aggregates them (protocol breakdown, top talkers, connection counts), and updates a simplified textual/graphical summary through the Python interface.

Outcome: The user gets an at-a-glance picture of network activity without reading raw Zeek logs directly.

4.3 Anomaly & Suspicious-Activity Alerting

User action: Enables alerting for a session and reviews the list of flagged events.

System behaviour: Uses Zeek's available detection and analysis capabilities, together with selected project-specific detection logic where appropriate, to identify predefined suspicious traffic patterns and generates alert entries with a timestamp, source, and reason. Examples may include port-scan behaviour and other predefined suspicious network patterns.

Outcome: The user receives a prioritised, readable list of suspicious events instead of having to manually inspect every connection.

4.4 Session History, Storage & Export

User action: Browses past capture sessions, selects one, and exports it (pcap and/or Zeek logs) for offline analysis elsewhere.

System behaviour: Maintains a local index of past sessions and their associated files/metadata, and packages the selected session for export.

Outcome: The user can retrieve, review, and share any past capture without losing data between sessions.

5. PLAN OF WORK

The project will follow an **Agile** approach with short, incremental iterations. The following sprint structure represents the planned sequence of development activities.

Stage	Focus
Sprint 0 — Setup	Study Zeek and ESP32/Pi packet capture; finalise hardware; set up shared development environment and repository.

Sprint 1 — Backlog	Write user stories for each functional feature; define acceptance criteria; prioritise the backlog.
Sprint 2 — Design	Define component architecture (capture layer, Zeek integration layer, Python UI layer, storage layer) and the interfaces between them.
Sprint 3 — Core Implementation	Each member implements their owned functionality against the agreed interfaces.
Sprint 4 — Integration	Combine the four modules into a single pipeline: capture → Zeek analysis → summary/alerts → storage/export.
Sprint 5 — Testing	Unit-test individual modules; run end-to-end integration tests using sample and lab-generated traffic.
Sprint 6 — Validation & Refinement	Validate against user stories and acceptance criteria; fix defects; improve output readability; prepare for demonstration.

6. PRODUCT OWNERSHIP / TEAM RESPONSIBILITIES

Team Member	Primary Functional Ownership	Responsibilities	Expected Deliverable
Ankita S	Live Capture & Session Control	Set up the Raspberry Pi-based capture pipeline (with the ESP32 as an auxiliary component if the final hardware design requires it); integrate with Zeek's input; implement start/stop, duration, and filter controls.	Working capture module producing .pcap output and feeding Zeek.
Anusha Gupta	Real-Time Traffic Summary Dashboard	Parse Zeek's log output; aggregate statistics; build the live-updating summary view in the Python interface.	Functional dashboard reflecting live capture state.
Charan M	Anomaly & Suspicious-Activity Alerting	Configure/extend Zeek detection scripts; build alert generation and display logic.	Alert module flagging suspicious traffic patterns.
Vaghasiya Akshar Arvindbhai	Session History, Storage & Export	Build session storage/indexing; implement export to .pcap / Zeek log formats.	Session browser and export feature.

Supporting Responsibilities (Shared — Not Counted as Primary Ownership)

- UI/UX consistency across the dashboard, alerts view, and session browser — coordinated jointly by the team.
- Local storage/database schema design — shared between the capture and storage owners.
- Unit and integration testing — each member tests their own module; all members participate in end-to-end integration testing.
- Documentation (README, user guide, synopsis and reports) — shared across the team.
- GitHub repository management and any CI setup — shared, coordinated by the team.
- Basic security review of alerting logic and data handling — shared.

7. QUALITATIVE / NON-FUNCTIONAL CONTRIBUTIONS

- **Performance:** The capture and log-processing pipeline should keep up with live traffic on a typical small LAN/lab segment without noticeable lag in the summary dashboard.
- **Security:** Captured data may include sensitive traffic; it should be stored locally by default, with no automatic transmission off the device.
- **Usability:** The dashboard and alerts should be understandable to a user without prior Zeek or Wireshark expertise.
- **Reliability:** A capture session should not crash the device and should handle interface disconnects gracefully.
- **Maintainability:** Clear separation between the capture, analysis, UI, and storage layers, so each can be modified independently.
- **Testability:** Each module should be testable independently using pre-recorded .pcap files as input, without requiring live network hardware for every test run.

8. EXPECTED OUTCOME

A working, portable device that allows a user to connect to a network segment, run a defined capture session, view a live traffic summary, receive alerts on a set of common suspicious patterns, and store or export the session for further review — demonstrating the complete functional pipeline end to end, at a scope appropriate for a Software Engineering mini-project.

9. FUTURE SCOPE

- Support for multiple simultaneous capture interfaces.
- Additional or more targeted Zeek detection scripts for specific attack patterns.
- An onboard display (e.g., a small LCD) for a fully standalone, screen-based mode without needing a connected laptop/phone.
- A lightweight remote dashboard for viewing sessions from a separate device over the local network.
- Basic trend analysis across multiple stored sessions.